

Module: CMP-6057A/CMP7057A
Advanced Web Development

Assignment: 001 Implementing WebAI

Set by: Dr Cheng Wang (Cheng.C.Wang@uea.ac.uk)
Date set: 04 September 2025
Value: 40%

Date due: 12 January 2026 [15:00]
Returned by: 9 February 2026
Submission: Blackboard
Checked by: Dr Jeannette Chin

Learning outcomes

- Frontend Programming
 - able to use modern front-end frameworks and browser features to develop complex web user interfaces using WebAI
 - able to integrate and run ML models in the browser using TensorFlow.js
 - able to handle media inputs (webcam, file upload) and render results on canvas
 - able to apply performance techniques, data preprocessing, and postprocessing
- Professionalism
 - able to practice and enhance employability skills through web development, presentations and demonstrations.

Specification

Overview

For this individual assignment, you will implement WebAI using a pre-trained TensorFlow.js model.

Description

This open-ended assignment encourages creativity. It challenges you to build a browser-based web application that performs **real-time object detection** using **TensorFlow.js**. You will use TensorFlow.js pre-trained **COCO-SSD model** for object detection (model paths: <https://github.com/tensorflow/tfjs-models/tree/master/coco-ssd>). Use your imagination to create an interesting

application, whether it's for smart home monitoring, traffic monitoring, or any other concept you can envision.

Tasks:

Develop a single webpage (HTML, CSS, JavaScript) to allow real-time object detection in a specific context of application. This web page **must be named** as `index.html`

Minimum requirements

- Write **client-side** JavaScript to implement the application logic. This code must process media inputs using the TensorFlow.js COCO-SSD model to enable a specific, chosen application.
- Can load the COCO-SSD model and detect objects on at least one input mode (e.g., image upload or webcam or both).
- For each detected object, draw a bounding box and display its class label and confidence scores.
- Display the results in 'user-friendly' format on the `index.html` page
- Code a Node Express server to serve this `index.html` as a static page (i.e. place in a folder called "public").
- Provides basic instructions for handling errors (e.g., denied camera permissions).

Note: *Submissions that do not meet all minimum requirements may receive a maximum of 50/100.*

P1. Essential Features (40 marks):

- **Model Integration (10 marks)**
 - Load model at startup with a visible loading state
 - Warmup pass to pre-allocate tensors
 - Clean-up / dispose tensors and avoid leaks
 - Version/model meta-data panel (name, size)
- **Media Inputs (10 marks)**
 - Webcam stream with start/stop controls
 - Static image/video upload/drag-drop and rerun detection
 - Note: You are free to implement both above, but you don't have to!
- **Rendering & UI (10 marks)**
 - Canvas overlay with boxes, labels, confidences
 - Class filter toggle (show/hide selected classes)
 - Confidence threshold slider
- **Performance & Responsiveness (10 marks)**
 - FPS (frame-per-second) counter and simple timing display
 - Device/backend selector (CPU/WebGL; WebGPU optional)
 - Responsive layout (mobile/tablet/desktop)

Note: Marks will be awarded according to the difficulty, quality and sophistication of your implementation.

P2. Desirable features (40 marks):

Implement any combination of features below to earn up to a **maximum of 40 marks**. Features are grouped by difficulty; each feature can only be counted once. Marks will be awarded according to the quality of the implementation. The maximum number of desirable features you can choose from below is 7. You need to tell us which features you have attempted, and list them out in the user guide clearly.

Silver features (5 marks each):

- Screenshot button saves annotated frame (PNG) with timestamp
- Per-class colour legend and consistent colouring across frames
- Basic object count, by class (e.g., number of persons, cars). This should be useful for your application!
- Explainable UI (e.g., show top-k class scores per detection; confidences over time)
- Dynamic model picker: Load different versions of COCO-SSD model and compare their performance

Gold features (10 marks each):

- Region-of-Interest: detect only within a draggable polygon/rectangle
- Track-by-ID across frames (e.g., naive IoU/centroid matching)
- Batch image detection with a gallery view and CSV export of results
- Dynamic model picker: Use YOLO-TFJS models and compare their performance with COCO-SSD models

Diamond features (15 marks):

- Train a custom SSD or YOLO model for your own set of object categories specific to your application

P3. Demonstration (20%)

- Record your demonstration in a video:
 - Describe the media inputs and approach you have used in your implementation.
 - Provide a demo of your application and explain the object detection results.
 - Your video must be in .mp4 format and no more than 15 minutes long.

Deliverables

1. **Recorded demo video** (.mp4) with no more than 15 minutes to be submitted to Blackboard.
2. **Source code submission**
 - a. Source code must be submitted to CMP stugit repo.
 - b. A README file with clear setup, run, and build instructions must be provided

NOTE: Source code will not be marked. However, it will be used for verifying your work. Failure to submit the source code may result in a penalty being applied.

3. A single A4 page user guide (filename must be `userguide.docx`). Provide the URL to your CMP stugit repo in this user guide. **Note:** this guide will not be marked but it will help the makers to locate your source code in Stugit.

Relationship to formative assessment

This assignment depends heavily on the material covered in the formative lab exercises on: TensorFlow.js lab exercises

Resources

- TensorFlow.js COCO-SSD model: <https://github.com/tensorflow/tfjs-models/tree/master/coco-ssd>
- TensorFlow.js API: <https://js.tensorflow.org/api/latest/>
- Python version: https://www.tensorflow.org/hub/tutorials/tf2_object_detection
- YOLO models: <https://docs.ultralytics.com/integrations/tfjs/>

Marking Scheme

The marking scheme is outlined below:

Student ID			
Marker's name		Date:	
			Mark
Essential Features (40%)			
Model Integration			
Media Inputs			
Rendering & UI			
Performance & Responsiveness			
Desirable Features (40%)			
Demo Video (20%)			
Comments:			
Total:			